实验二 I/O 与 Memory-Mapped I/O

一、实验目的

完成本次实验之后，学生应能够：

1．理解 I/O 设备如何通过“内存映射（Memory-Mapped I/O, MMIO）”被 CPU 的 load/store 指令访问。

2．掌握根据外设手册（device manual）为设备分配 I/O 寄存器地址（I/O-mapped address），并进行读写访问。

3．建立 CPU-总线-存储器-外设的系统整体视角；理解地址空间（address space）、地址译码（address decode）、总线（bus）与存储层次（hierarchy）与 I/O 映射之间的关系。

4．能够通过 MMIO 控制板载外设（LED、开关/按键、UART0），实现可观测、可交互的系统功能。

二、知识背景与关键概念

1. MMIO 的核心思想

- 在 MMIO 中，外设控制/状态寄存器被映射到 CPU 的统一地址空间。CPU 对某个“物理地址”的读写，可能被总线译码到：
  - 存储器（SRAM/DDR/Flash 等），或
  - 外设寄存器（GPIO、UART 等）。
- 外设寄存器与普通内存变量的差异在于：
  - 外设寄存器读写通常具有其他影响（例如写 TXDATA 会触发发送；读 STATUS 会清除标志位等）。
  - 外设区域往往需要非缓存（non-cacheable）/强顺序（strongly ordered）语义（依具体平台实现而定）。

2. MMIO vs. Port I/O（独立 I/O 空间）

MMIO：I/O 与内存共享同一地址空间，使用普通 load/store 指令访问。

Port I/O（PMIO）：I/O 使用独立端口地址空间，需要专用指令（典型如 x86 in/out）。

三、预习要求

在实验前，学生应完成以下准备工作：

- 阅读教材 **Computer Organization and Architecture: Performance Design (Stallings, 10th Edition)** 中关于 I/O、I/O 映射（I/O-mapped I/O / port I/O）与内存映射 I/O（MMIO）的章节。理解地址空间（address space）、I/O 寄存器（I/O registers）、内存与外设共享地址空间的思想。
- 查阅与理解 MMIO 的概念、优缺点，以及与端口（Port I/O）/独立 I/O 空间（PMIO）的区别。
- 熟悉本实验平台：Nuclei DDR200T FPGA Evaluation Kit（以下简称 DDR200T 板）与 Hummingbirdv2 E203（HBirdv2 E203）SoC。查阅其用户手册/技术文档，了解其总线接口（GPIO、UART、JTAG etc.）以及板上外设（LED、开关、UART0、GPIO...）的映射关系。
- 准备好实验环境：安装并熟悉 Nuclei Studio IDE，了解如何使用 debugger / JTAG / UART 等进行烧写与调试。

四、实验环境与设备

- 硬件平台：Nulcei DDR200T FPGA Evaluation Kit，已经加载包含 Hummingbirdv2 E203 SoC 的 bitstream。
- 软件&开发工具：Nuclei Studio，支持 C（或汇编）程序编译与调试；Hummingbird debugger + JTAG + UART 接口用于下载程序、观察 UART 输出 / GPIO 状态。
- 参考资料/手册：E203 SoC 文档（MMIO 寄存器地址定义、外设映射），DDR200T 板说明书（GPIO、LED、开关引脚分配），以及教材或其他 I/O / cache /体系结构章节。

五、实验内容与任务概览

学生将通过实际操作理解并实现通过 MMIO 访问外设（LED、开关、GPIO 等），并进行状态读写。

任务 1：理解与配置 I/O 映射（I/O 寄存器地址）

目的：从文档/SDK 中找出外设寄存器基地址、寄存器偏移、关键位域，并在代码中形成可维护的宏/封装。

1. 需查找并填写（至少包含）：

LED 所连接 GPIO 外设的 MMIO 基地址与输出寄存器偏移（以及是否需要方向寄存器 DIR/EN）

开关/按键输入寄存器地址（或 GPIO IN 寄存器）

UART0 的 MMIO 基地址与关键寄存器：

- TXDATA（发送数据）
- RXDATA（接收数据）
- STATUS/STATE（包含 TX ready / RX valid 等标志）

2. 代码交付物（必须）：

在代码中用 #define 或头文件常量定义地址与偏移，例如：

```
#define LED_BASE_ADDR   0xXXXXXXXXu
#define LED_OUT_REG     (LED_BASE_ADDR + 0x00u)
#define SW_BASE_ADDR    0xYYYYYYYYu
#define SW_IN_REG       (SW_BASE_ADDR + 0x00u)
#define UART0_BASE_ADDR 0xZZZZZZZZu
#define UART0_TX_REG    (UART0_BASE_ADDR + 0x00u)
#define UART0_STAT_REG  (UART0_BASE_ADDR + 0x04u)
```

采用 volatile 访问寄存器，或提供 mmio_read32/mmio_write32 封装

定义本实验用到的状态位掩码（mask）

3. 报告交付物（必须）：MMIO 映射表

报告中必须包含一张表

外设	基地址	寄存器名	Offset	R/W 属性	含义
GPIO (LED)	0x.....		0x...		
GPIO (SW)	0x.....		0x...		
UART0	0x.....		0x...		
UART0	0x.....		0x...		
UART0	0x.....		0x...		

任务 2：通过 MMIO 进行外设读写控制（LED/开关联动）

在任务 1 的基础上，编写程序实现：

1. LED 测试：

- 周期性地按一定顺序点亮/熄灭多个 LED（如“流水灯”）。
- 体会对某个 MMIO 地址写入不同 bit pattern 对硬件引脚电平的直接影响。

2. 开关与 LED 的联动：

- 读取开关/按键的状态寄存器。
- 将开关的 bit 状态映射到 LED 显示（例如 SW[0]决定 LED[0]是否亮）。

六、实验步骤

1. 环境准备

（1）硬件连接与准备

- 将 DDR200T 开放板接入电源（参考板子说明书：DDR200T 通常使用外接 12V 电源）。
- 用官方 Debugger（JTAG）接口（Hummingbird-Debugger）连接 PC 与板子上的 MCU-JTAG socket，以便下载程序和调试。
- 将 UART0 的串口（USB-串口线 / 转换器）连接到 PC，以便观察终端输出和输入字

符控制。

（2）在 PC 上启动 Nuclei Studio。确认 IDE 可以识别调试器 / 板子（JTAG）并正常使用。

2. 创建工程（基于 Nuclei Studio）

以下步骤参考官方用户指南 / Quick Start 文档。

（1） 新建项目

- 在 Nuclei Studio 中选择 File -> New -> C/C++ Project。
- 在弹出对话框中，选择“Hello World RISC-V C Project”作为项目类型。
- 完成项目创建。

（2） 编译（Build）项目

- 点击“Build”按钮 / menu -> Build Project。若无错误，继续下一步。

（3） 下载（flash / load）程序到板子

- 使用 ILM 模式，通过 OpenOCD + GDB 将程序写入板子。

（4） 启动串口终端（Serial Terminal）

- 在 Nuclei Studio 中：Window -> Show View -> Terminal，点击“Open a Terminal”按钮，选择“Serial Terminal”。
- 设置串口号（对应连接到 UART0 的串口设备），波特率/数据位/停止位等（与 UART0 初始化设定保持一致）。

（5） 重置（Reset）MCU / 板子，观察串口终端（或 LED/外设）的输出/响应，确认基础环境（LED，UART，GPIO）正常。

备注：这部分流程等同“Hello World Demo”的运行方式，官方 Quick-Start 文档中已有详细演示。

3. 任务

（1）任务 1：确认 MMIO 映射并建立寄存器访问接口

- **查手册定位外设基地址与寄存器偏移**
  - **GPIO：GPIOA 基地址为 0x1001_2000（GPIOB：0x1004_0000）。**
    - 寄存器 offset（32-bit 寄存器，bit 对应 pin）：

- GPIO_PADDIR：0x00（方向）
- GPIO_PADIN：0x04（输入值）
- GPIO_PADOUT：0x08（输出值）
- GPIO_IOFCFG：0x1C（SW/IOF 复用选择）

◆ SW/IOF 模式原则：GPIO_IOFCFG 对应 bit=0 表示该 pin 处于 SW 控制（普通 GPIO），bit=1 表示 IOF 控制（被外设复用）。

■ **UART0（用于串口输出/调试）。**

■ UART0 基地址：0x1001_3000。

■ 关键寄存器 offset：

◆ UART_RBR/UART_THR：0x00（接收/发送共享地址，取决于读/写与 DLAB）

◆ UART_LSR：0x14（Line Status Register）

◆ TX ready：LSR.THRE（bit5，TX FIFO empty）

◆ RX valid：LSR.DR（bit0，收到数据）

■ **建立本实验专用地址/位定义（建议新建头文件）**

■ 新建 lab2_mmio.h（或同类文件），集中放置：

◆ GPIO/LED/SW/UART0 的 base、offset、mask

◆ 读写封装 mmio_read32/mmio_write32 或 volatile 宏

◆ UART LSR 的状态位 mask（THRE、DR）

◆ LED/SW/Button 的 bit mask：优先直接复用 board_ddr200t.h 中的 SOC_*_GPIO_MASK

■ 在 main.c 引入该头文件，避免在逻辑代码中散落硬编码地址。

■ 推荐：lab2_mmio.h 最小模板

```
#ifndef APPLICATION_LAB2_MMIO_H_
#define APPLICATION_LAB2_MMIO_H_
#pragma once
#include <stdint.h>
#include "hbird_sdk_soc.h"
#include "board_ddr200t.h"
// GPIO offsets
#define LAB_GPIO_PADDIR_OFS   0x00u
#define LAB_GPIO_PADIN_OFS    0x04u
#define LAB_GPIO_PADOUT_OFS   0x08u
#define LAB_GPIO_IOFCFG_OFS   0x1Cu
```

```
// UART offsets
#define LAB_UART_RBR_THR_OFS  0x00u
#define LAB_UART_LSR_OFS      0x14u
// Addresses (reuse SDK-provided GPIOA_BASE / UART0_BASE)
#define LAB_GPIOA_PADDIR_ADDR (GPIOA_BASE + LAB_GPIO_PADDIR_OFS)
#define LAB_GPIOA_PADIN_ADDR  (GPIOA_BASE + LAB_GPIO_PADIN_OFS)
#define LAB_GPIOA_PADOUT_ADDR (GPIOA_BASE + LAB_GPIO_PADOUT_OFS)
#define LAB_GPIOA_IOFCFG_ADDR (GPIOA_BASE + LAB_GPIO_IOFCFG_OFS)
#define LAB_UART0_RBR_ADDR    (UART0_BASE + LAB_UART_RBR_THR_OFS)
#define LAB_UART0_THR_ADDR    (UART0_BASE + LAB_UART_RBR_THR_OFS)
#define LAB_UART0_LSR_ADDR    (UART0_BASE + LAB_UART_LSR_OFS)
// UART status bits (LSR)
#define LAB_UART_LSR_DR       (1u << 0)  // RX valid
#define LAB_UART_LSR_THRE     (1u << 5)  // TX ready
// MMIO access helpers (reuse SDK REG32)
static inline uint32_t mmio_read32(uintptr_t addr) { return REG32(addr);
 }
static inline void mmio_write32(uintptr_t addr, uint32_t v) { REG32(add
r) = v; }
......
#endif /* APPLICATION_LAB2_MMIO_H_ */
```

- 完成“MMIO 映射表”（报告必填）
  - 按实验手册要求填写“外设-寄存器-offset”表格
  - 确认至少能解释：UART 的 TX ready/RX valid 位为何需要轮询等待。
- 下载运行，做最小验证
  - 写一个最小程序：启动后通过 UART 打印

    `MMIO & UART Lab Start\r\n`
  - 能看到该字符串输出则说明：下载、复位、串口链路基本正确。

（2）任务 2：GPIO 控制（流水灯+开关联动）

- 前置条件（必须满足）
  - 你已能通过串口看到启动信息（例如 MMIO & UART Lab Start\r\n）。
  - 你已在 lab2_mmio.h 中完成 GPIOA 的 base+offset 映射，并能通过 mmio_read32/mmio_write32 访问寄存器。
  - 你已确认 GPIO 寄存器与基地址（用于报告“MMIO 映射表”）：
    - GPIOA base：0x1001_2000；GPIOB base：0x1004_0000

- GPIO_PADDIR=0x00 ， GPIO_PADIN=0x04 ， GPIO_PADOUT=0x08 ， GPIO_IOFCFG=0x1C
- 32-bit 寄存器，每个 bit 对应一个 pin
- 注意：GPIO_IOFCFG 对应 bit = 0 表示该 pin 处于 SW 控制（普通 GPIO）；bit = 1 表示 IOF 控制（被外设复用）。

- 子任务 A：实现 LED 流水灯
  - GPIO 初始化（必须）
    - 在 main.c 启动后第一时间调用 lab_gpio_init_ddrt200t()（或你们自己封装的同等函数），确保：
      - LED/RGB 对应 bit 的 IOFCFG 清 0（切回 SW 控制）
      - LED/RGB 对应 bit 的 PADDIR 置 1（输出）
      - PADOUT 初始关闭 LED（按需要）
  - 翻转 LED0（read-modify-write）
    - 读 GPIO_PADOUT
    - 对 SOC_LED_0_GPIO_MASK 做 XOR
    - 写回 GPIO_PADOUT
  - 软件延时；
    - 允许用简单 busy-wait（任务 2 不强制用定时器）
    - LED0 以固定频率闪烁
  - 接下来 LED0 -> LED5 依次点亮（或来回跑马），周期可配置。
  - 实现建议：
    - 维护一个 uint32_t led_pattern，只在 SOC_LED_GPIO_MASK 范围内移动。
    - 每次更新 PADOUT 时必须保留非 LED 位（尤其不要误伤 RGB 或其他 GPIO）：
      - padout = (padout & ~SOC_LED_GPIO_MASK) | (led_pattern & SOC_LED_GPIO_MASK);
- 子任务 B：实现开关/按键与 LED 联动（SW[0...5] -> LED[0...5])
  - 确认 SW/按键引脚处于输入模式
    - IOFCFG：SW/Button 对应 bit 清 0 （SW 控制）
    - PADDIR：SW/Buttopn 对应 bit 清 0 （输入）
  - 读取输入寄存器

- 从 GPIO_PADIN 读取当前输入值（不是 PADOUT）。

- 做位映射（推荐写法：利用连续 bit 区间）
  - DDR200T 的 SW0...SW5 在 GPIOA 的 bit[26...31]（由 board_ddr200t.h 给出的 offset/mask）。
  - LED0...LED5 在 bit[20...25]
  - 因此可用“右移 6 位”直接对齐（26 -> 20）：
    - led_bits = (padin & SOC_SW_GPIO_MASK) >> 6;
    - padout = (padout & ~SOC_LED_GPIO_MASK) | (led_bits & SOC_LED_GPIO_MASK);
- 循环刷新
  - 推荐每 50~100ms 刷新一次，防止串口输出或其他逻辑造成“看起来不同步”。
- 验收点（自检）
  - 任意拨动 SW0...SW5，LED0...LED5 立即正确显示；
  - 报告中解释：为什么读输入要用 PADIN，写输出要用 PADOUT；为什么必须先配置 IOFCFG/PADDIR。

（3）收尾与提交检查（提交前自检）

- 代码规范性：
  - 地址、offset、mask 不散落硬编码，集中在头文件中；
  - 访问寄存器使用 volatile 或 mmio_read32/mmio_write32；
  - 关键状态位（UART）使用清晰命名的宏。
- 运行证据：
  - LED 流水灯现象（照片或视频截图）；
  - 串口终端交互截图（包含输入与输出）。
- 报告完整性：
  - MMIO 映射表填写完整；
  - 至少一条排障记录；
  - 思考题回答完整。

七、实验记录与报告要求

提交物：

源码工程（含你自己实现的关键文件）

实验报告（PDF）

报告必须包含：

1. 实验环境：板卡、SoC/CPU、工具链/IDE 版本、串口参数
2. MMIO 映射表（必选项）
3. 关键实现说明

   寄存器定义与访问封装

   LED 流水灯/联动逻辑

   UART 轮询收发与命令解析
4. 实验现象证据：

   LED 流水灯照片/截图

   串口交互截图（包含输入命令与输出）
5. 问题与排查记录，从现象->定位->解决
6. 结论：总结你对 MMIO 的理解，与 PMIO 对比，volatile 的作用等。

八、思考题

1. 为什么访问 MMIO 寄存器通常需要 volatile？不用会出现哪些可观测问题？
2. MMIO 与 Port I/O（独立 I/O 空间）在指令、地址空间、软件移植性方面各有哪些优缺点？

RISC-V 为何通常采用 MMIO？

3. UART 发送为何要等待 TX ready？如果不等待可能出现什么现象（覆盖/丢字节/阻塞）？
4. 为什么外设寄存器区域通常不应该被缓存？若被缓存可能产生何种错误行为？

九、参考文档

```
HBirdV2 SoC 文档 PDF:
https://doc.nucleisys.com/hbirdv2/hbirdv2_soc.pdf
HBirdV2 SoC 在线文档首页:
https://doc.nucleisys.com/hbirdv2/
```

```
HBirdV2 SoC 外设寄存器章节（含寄存器表）:
https://doc.nucleisys.com/hbirdv2/soc_peripherals/ips.html
HBirdV2 E203 开源仓库:
https://github.com/riscv-mcu/e203_hbirdv2
HBird SDK（常含地址/寄存器宏定义与示例）:
https://github.com/riscv-mcu/hbird-sdk
```

十、附录

1. MMIO 寄存器访问模板

```c
#include <stdint.h>
static inline void mmio_write32(uintptr_t addr, uint32_t val) {
    *(volatile uint32_t *)addr = val;
}
static inline uint32_t mmio_read32(uintptr_t addr) {
    return *(volatile uint32_t *)addr;
}
/* 可选：若平台/课程需要更强顺序语义，可引入 fence（进阶，不强制）
static inline void mmio_fence(void) {
    __asm__ volatile ("fence iorw, iorw" ::: "memory");
}
*/
```

2. 地址与寄存器定义

以下地址、offset、mask 必须来自 DDR200T + E203 官方文档或课程 SDK 头文件，请自行填写。

```c
#define GPIO_BASE_ADDR       0x________u
#define GPIO_OUT_OFFSET      0x__u
#define GPIO_IN_OFFSET       0x__u
#define GPIO_DIR_OFFSET      0x__u   /* 如需要 */
#define UART0_BASE_ADDR      0x________u
#define UART_TXDATA_OFFSET   0x__u
#define UART_RXDATA_OFFSET   0x__u
#define UART_STATUS_OFFSET   0x__u
#define UART_TXRDY_MASK      (1u << __)  /* 待填写 */
#define UART_RXVAL_MASK      (1u << __)  /* 待填写 */
#define GPIO_OUT_REG_ADDR    (GPIO_BASE_ADDR + GPIO_OUT_OFFSET)
#define GPIO_IN_REG_ADDR     (GPIO_BASE_ADDR + GPIO_IN_OFFSET)
#define GPIO_DIR_REG_ADDR    (GPIO_BASE_ADDR + GPIO_DIR_OFFSET)
#define UART_TXDATA_ADDR     (UART0_BASE_ADDR + UART_TXDATA_OFFSET)
```

```
#define UART_RXDATA_ADDR      (UART0_BASE_ADDR + UART_RXDATA_OFFSET)
#define UART_STATUS_ADDR      (UART0_BASE_ADDR + UART_STATUS_OFFSET)
```

3. 常见故障与排查清单

- LED 不亮/全亮：
  - GPIO 方向未设为输出；
  - LED 为低电平点亮但逻辑未取反；
  - OUT 寄存器/位定义错误。
- UART 乱码：
  - 波特率/8N1 不匹配；
  - 时钟源/分频与 SDK 默认不一致；
  - 终端串口号选错。
- 开关读值恒 0：
  - 引脚复用未切到 GPIO；
  - IN 寄存器/offset 错；
  - 外设时钟未使能或未从复位释放（依平台）。
- 程序跑飞/异常：
  - 访问不存在的 MMIO 地址触发总线异常；
  - 访问宽度不匹配/未对齐。
- REG |= mask 无效或引发异常行为：
  - 寄存器为 W1C/W1S 等特殊语义；需按手册改写为“写特定位值”而非读改写。